Kmeans Text Clustering

1. Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
```

Purpose

TfidfVectorizer

Converts text documents into numerical feature vectors using TF-IDF.

KMeans

Implements the K-Means clustering algorithm.

2. Sample Documents

```
documents = [
    "machine learning and artificial intelligence",
    "deep learning using neural networks",
    "natural language processing techniques",
    "football world cup tournament",
    "cricket match and batting skills",
    "soccer players and championship"
]
```

The dataset contains six documents.

Technology-Related Documents

```
machine learning and artificial intelligence
deep learning using neural networks
natural language processing techniques
```

Sports-Related Documents

```
football world cup tournament
cricket match and batting skills
soccer players and championship
```

Although these categories exist conceptually, the algorithm is **not given these labels**. It must discover groups automatically.

3. Convert Text into TF-IDF Vectors

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

Why?

Machine learning algorithms require numerical data.

The vectorizer converts each document into a TF-IDF vector.

Example

Document:

```
deep learning using neural networks
```

becomes a numerical representation such as:

```
[0.12, 0.45, 0.67, ...]
```

where each value represents the importance of a word.

4. What is TF-IDF?

Term Frequency (TF)

Measures how often a word appears in a document.

Inverse Document Frequency (IDF)

Measures how unique a word is.

Where:

- N = Total documents
- DF = Number of documents containing the term

TF-IDF

Important words get larger weights.

Examples:

```
intelligence
neural
football
cricket
```

Common words get lower weights.

5. Define Number of Clusters

```
k = 2
```

Meaning

The model should create:

```
2 clusters
```

In this dataset, the expected groups are:

```
Cluster 0 → Technology
Cluster 1 → Sports
```

or vice versa.

The cluster numbers themselves have no meaning; only the grouping matters.

6. Create K-Means Model

```
model = KMeans(
    n_clusters=k,
    random_state=42
)
```

Parameters

n_clusters=k

Specifies the number of groups:

```
n_clusters = 2
```

random_state=42

Ensures reproducible results every time the program runs.

7. Train the Model

```
model.fit(X)
```

What Happens Internally?

K-Means follows these steps:

Step 1: Select Initial Centroids

Randomly choose:

```
Cluster Center 1
Cluster Center 2
```

Step 2: Assign Documents

Each document is assigned to the nearest centroid.

Step 3: Recompute Centroids

Calculate new cluster centers.

Step 4: Repeat

Continue until the clusters stop changing.

8. Understanding K-Means

Suppose documents are represented as points:

```
Technology
     *
   *   *
            Sports
          *  *  *
```

K-Means finds cluster centers and groups nearby points together.

Result:

```
Cluster 0
 ├─ AI document
 ├─ Deep Learning document
```

```
    └─ NLP document

Cluster 1
    ├─ Football document
    ├─ Cricket document
    └─ Soccer document
```

9. Display Cluster Assignments

```
for i in range(len(documents)):
```

Loops through all documents.

```
print("Document:", documents[i])
print("Cluster :", model.labels_[i])
```

model.labels_ contains the cluster number assigned to each document.

10. Possible Output

```
Document: machine learning and artificial intelligence
Cluster : 0

Document: deep learning using neural networks
Cluster : 0

Document: natural language processing techniques
Cluster : 0

 Document: football world cup tournament
 Cluster : 1

 Document: cricket match and batting skills
 Cluster : 1

 Document: soccer players and championship
 Cluster : 1
```

The cluster numbers may swap:

```
Technology → Cluster 1
Sports     → Cluster 0
```

This is normal in K-Means clustering.

Clustering vs Classification

Feature	Classification	Clustering
Labels Provided	Yes	No
Learning Type	Supervised	Unsupervised
Goal	Predict category	Discover groups
Example Algorithm	SVM, Naive Bayes	K-Means

Why Use K-Means for Text?

K-Means helps:

- Organize large document collections
- Discover hidden topics
- Group similar news articles
- Cluster customer reviews
- Segment search results
- Topic modeling (basic form)

Real-World Applications

News Websites

Group articles into:

```
Politics
Sports
Technology
Business
```

E-commerce

Cluster product reviews into:

```
Positive
Negative
Neutral
```

Search Engines

Group similar search results together.

Social Media

Identify trending discussion topics.

Program Workflow

```
Text Documents
       ↓
TF-IDF Vectorization
       ↓
Numerical Features
       ↓
K-Means Algorithm
       ↓
Cluster Formation
       ↓
 Assign Documents to Clusters
       ↓
 Display Results
```

Key Concepts Used

Concept	Purpose
TF-IDF	Convert text into numerical vectors
K-Means	Cluster similar documents
Centroid	Center point of a cluster
Unsupervised Learning	Learning without labels
Text Clustering	Grouping similar documents

Interview Questions

Q1. Why is K-Means called an unsupervised algorithm?

Answer: Because it does not require labeled training data. It automatically discovers patterns and groups in the data.

Q2. What does n_clusters=2 mean?

Answer: The algorithm will divide the documents into exactly two groups.

Q3. Why is TF-IDF used before K-Means?

Answer: K-Means works on numerical vectors, so TF-IDF converts text documents into meaningful numeric representations.

Q4. What is a centroid?

Answer: A centroid is the center point of a cluster. K-Means assigns documents to the nearest centroid.

Time Complexity

Let:

- **N** = Number of documents
- **K** = Number of clusters
- **I** = Number of iterations
- **D** = Number of features (vocabulary terms)

K-Means Complexity

TF-IDF Creation

For small and medium-sized text datasets, K-Means is fast and highly effective.

Key Takeaway

This program performs **text clustering** by first converting documents into **TF-IDF vectors** and then applying **K-Means** to automatically group similar documents. Since no labels are provided, it is an example of **unsupervised learning**, commonly used for document organization and topic discovery.